

RESUMO DO MÓDULO

Métodos de Resposta

No Express.js, o objeto de resposta (res) possui vários métodos que permitem enviar uma resposta HTTP ao cliente e encerrar o ciclo de requisição e resposta. Esses métodos são essenciais para personalizar as respostas que serão enviadas aos clientes, definindo o conteúdo, o código de status e outras propriedades da resposta.

Se nenhum desses métodos for chamado, a solicitação do cliente ficará pendente e não será concluída corretamente. Aqui estão os métodos mais comuns e como utilizá-los no Express.

`res.send()`

O método `res.send()` envia uma resposta HTTP com um corpo de resposta. Ele detecta automaticamente o tipo de conteúdo com base no argumento fornecido e define os cabeçalhos apropriados. Você pode enviar uma string, um objeto JSON ou até mesmo um buffer usando `res.send()`.

Aqui está um exemplo básico usando `res.send()` para enviar uma resposta simples com uma string:

javascript

```
const express = require('express');
```

```
const app = express();
```

```
app.get('/', (req, res) => {
```

```
  res.send('Bem-vindo ao meu site!');
```

```
});
```

```
app.listen(3000, () => {  
  console.log('Server listening on port 3000');  
});
```

Nesse exemplo, quando a rota raiz (/) é acessada, o método `res.send()` é usado para enviar a resposta com a string "Bem-vindo ao meu site!".

```
res.json()
```

O método `res.json()` envia uma resposta HTTP com um corpo no formato JSON. Ele converte automaticamente o objeto passado em uma string JSON e define os cabeçalhos apropriados para indicar o tipo de conteúdo.

Por exemplo, suponha que você tenha uma rota que retorna um usuário específico pelo seu ID:

```
javascript
```

```
const express = require('express');  
const app = express();
```

```
const users = [  
  { id: 1, name: 'John Doe' },  
  { id: 2, name: 'Jane Doe' },  
  { id: 3, name: 'Bob Smith' }  
];
```

```
app.get('/users/:id', (req, res) => {  
  const id = req.params.id;  
  const user = users.find(u => u.id == id);  
  
  return res.json(user);  
});  
  
app.listen(3000, () => {  
  console.log('Server listening on port 3000');  
});
```

Neste exemplo, a rota `/users/:id` recebe um ID de usuário na URL e retorna o usuário correspondente se ele existir no array `users`.

```
res.status()
```

O método `res.status()` define o código de status HTTP da resposta. Ele pode ser encadeado com outros métodos de resposta, como `res.send()` ou `res.json()`, para enviar uma resposta com o código de status apropriado.

Aqui está um exemplo em que o método `res.status()` é usado para definir um código de status 404 (não encontrado) quando um usuário não é localizado:

```
javascript
```

```
const express = require('express');
```

```
const app = express();
```

```
const users = [
```

```
  { id: 1, name: 'John Doe' },
```

```
  { id: 2, name: 'Jane Doe' },
```

```
  { id: 3, name: 'Bob Smith' }
```

```
];
```

```
app.get('/users/:id', (req, res) => {
```

```
  const id = req.params.id;
```

```
  const user = users.find(u => u.id == id);
```

```
  if (!user) {
```

```
    return res.status(404).send('User not found');
```

```
  }
```

```
  return res.json(user);
```

```
});
```

```
app.listen(3000, () => {
```

```
  console.log('Server listening on port 3000');
```

```
});
```

Nesse caso, se o usuário não for encontrado, a rota retorna um status 404 com a mensagem "User not found".

```
res.download()
```

O método `res.download()` é utilizado para enviar um arquivo para o cliente como uma resposta. Isso permite que o usuário faça o download do arquivo para o seu dispositivo.

Por exemplo, para enviar um arquivo PDF ao acessar a rota `/download`:

javascript

```
app.get('/download', (req, res) => {  
  const file = __dirname + '/arquivo.pdf';  
  res.download(file);  
});
```

Ao acessar `/download`, o arquivo PDF é enviado para o cliente. Você também pode definir um nome personalizado para o arquivo durante o download, passando-o como segundo parâmetro:

javascript

```
app.get('/download', (req, res) => {  
  const file = __dirname + '/arquivo.pdf';  
  res.download(file, 'relatorio.pdf');  
});
```

Nesse caso, o arquivo será baixado com o nome "relatorio.pdf" em vez do nome original do arquivo.

`res.redirect()`

O método `res.redirect()` envia uma resposta HTTP que redireciona o cliente para outra URL. Você pode especificar a URL de destino como argumento:

javascript

```
app.get('/', (req, res) => {  
  res.redirect('/users/ricardo');  
});
```

Ou ainda redirecionar para uma URL externa:

javascript

```
app.get('/', (req, res) => {  
  res.redirect('http://example.com');  
});
```

Você também pode passar um código de status de redirecionamento (como 301 para redirecionamento permanente) como o primeiro argumento:

javascript

```
app.get('/', (req, res) => {  
  res.redirect(301, 'http://example.com');  
});
```

Resumo

Os métodos de resposta do Express.js permitem que você personalize as respostas enviadas ao cliente, definindo:

- O corpo (res.send(), res.json()).
- O código de status (res.status()).
- Redirecionamentos (res.redirect()).
- Download de arquivos (res.download()).

Cada um desses métodos desempenha um papel fundamental na criação de respostas dinâmicas e personalizadas, permitindo que o servidor atenda a diferentes tipos de solicitações e forneça a resposta adequada para cada caso.

Entender e utilizar esses métodos corretamente é essencial para construir aplicativos web robustos e eficientes com Express.
